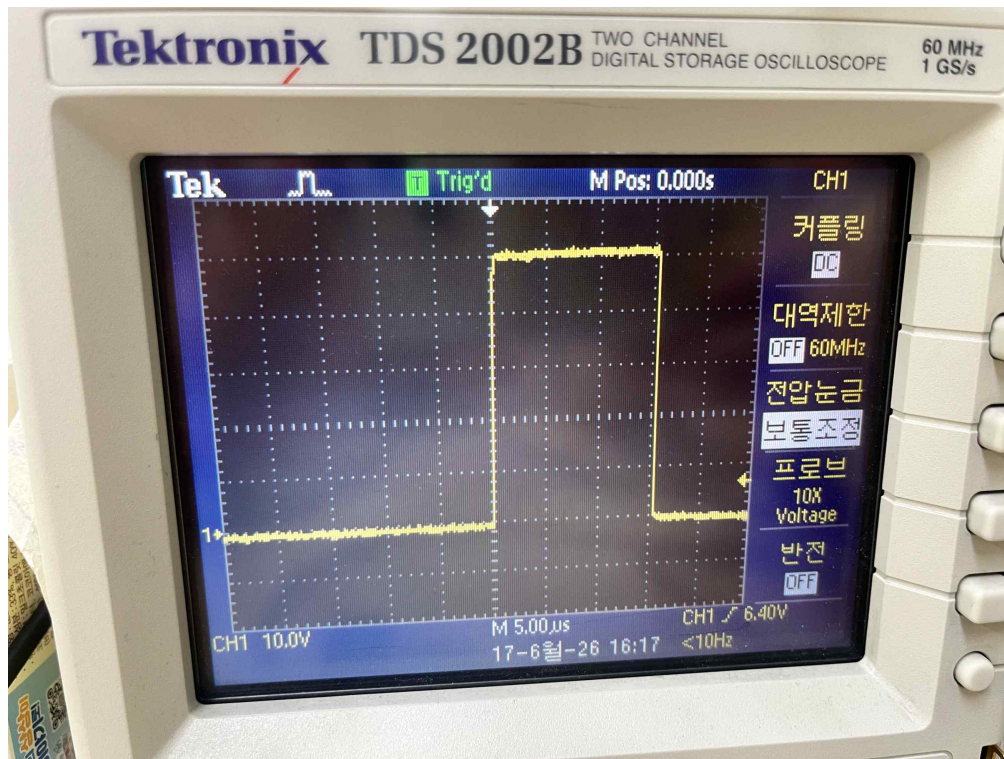


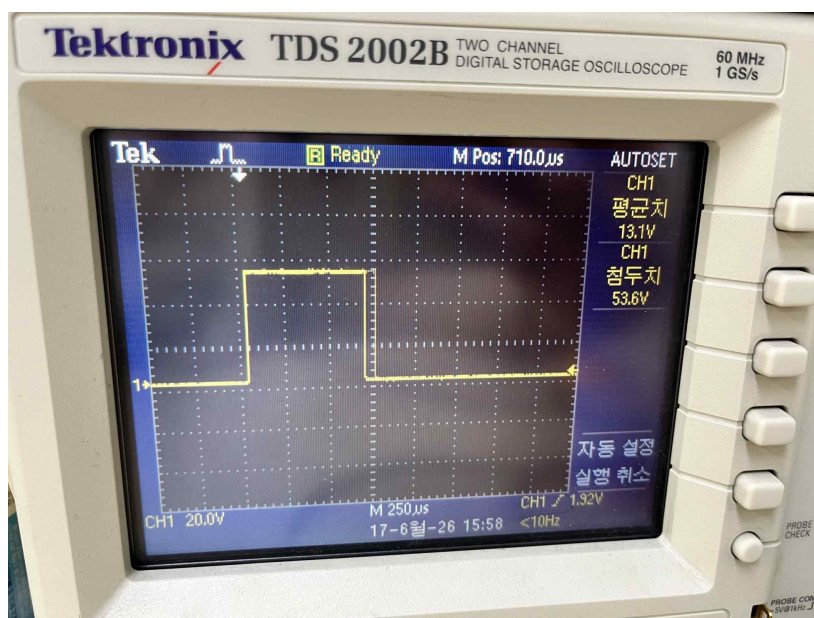
박수민_0617_과제

2. 오실로스코프 파형 측정

2-1. trigger pulse 측정



2-2. 거리파형



3. 거리측정 LED 연동

```
#ultrasonic.c
/*
 * ultrasonic.c
 *
 * Created: 2026-06-17 오후 1:32:59
 * Author: kccistc
 */

#include "ultrasonic.h"

void init_ultrasonic(void);
void make_trigger(void);
void ultrasonic_processing(void);
void led_distance_control(int dist_cm);

extern volatile int ultrasonic_check_time;

volatile int ultrasonic_distance = 0;
volatile char scm[50];

// p.278 표 12-3
// INT4: PE4 외부(ext) INT4 초음파 센서 상승.하강 엣지 둘다 이곳으로 들어옴
// 결국 상승 엣지 때 한번, 하강 엣지 때 한번 들어옴
// 0x000A
ISR(INT4_vect)
{
    // 1. 상승 엣지
    if (ECHO_PORT & (1 << ECHO_PIN))
    {
        TCNT1 = 0;

    }
    // 2. 하강 엣지
    else
    {
        // TCNT1: timer count 1
        // ex. TCNT1에 10 들어있다고 가정해보자.
        // 15.625KHz의 1주기: 64μs
        // 0.000064sec * 10 => 0.00064sec(640μs)
        // 640μs/58 (1cm 이동하는데 소요되는 시간) : 11cm
        // 1sec: 1000000μs
    }
}
```

```

        ultrasonic_distance = TCNT1 * 1000000.0 * 1024/ F_CPU;

        // ---- 소요 시간을 cm로 환산 ----
        sprintf((char *)scm, "dis: %dcm\n", ultrasonic_distance/58);
    }

}

void init_ultrasonic(void)
{
    TRIG_DDR |= 1 << TRIG_PIN;          // output mode로 설정
    ECHO_DDR &= ~(1 << ECHO_PIN); // input mode로 설정

    // p.289 표 12-6 p.288 그림 12-8 참조
    // 0 1: 상승에지, 하강에지 둘다 INT 를 띄우도록 요청한다.
    EICRB |= 0 << ISC41 | 1 << ISC40;

    // 16bit timer/counter 1번을 사용하기로 하자.
    // timer int를 사용하지 않는다.
    // 16bit timer1 16bit로 표시할 수 있는 최대값 65535(max)
    // 16MHz / 1024 분주: 16000000Hz/1024 --> 15625Hz -> 15.625KHz
    // 1주기(1개의 펄스 소요시간) : T=1/f = 1/15625 --> 0.000064sec => 64μs
    // 분주비 1024로 설정, p.318 표 14-1
    TCCR1B |= 1 << CS12 | 1 << CS10;      // 1024분주

    //----EINT4(external interrupt 4번) 설정----
    // p.287 그림 12-6
    EIMSK |= 1 << INT4;          // 외부 INT4(ECHO핀)

}

void make_trigger(void)
{
    TRIG_PORT &= ~(1 << TRIG_PIN); // low로 만든다.
    _delay_us(1);
    TRIG_PORT |= (1 << TRIG_PIN);  // high로 만든다.
    _delay_us(15);                  //          규격에는          10μs인데
    redundancy로
    TRIG_PORT &= ~(1 << TRIG_PIN); // low로 만든다.

}

void ultrasonic_processing(void)

```

```

{
    if (ultrasonic_check_time >= 1000)          // 1초
    {
        ultrasonic_check_time = 0;              // 초기화
        printf("%s", scm);
        make_trigger();

        int dist_cm = ultrasonic_distance / 58;
        led_distance_control(dist_cm);
    }
}

void led_distance_control(int dist_cm)
{
    if (dist_cm <= 2)
        PORTA = 0b00000001; // led 0
    else if (dist_cm <= 3)
        PORTA = 0b00000011; // led 0,1
    else if (dist_cm <= 4)
        PORTA = 0b00000111; // led 0,1,2
    else if (dist_cm <= 5)
        PORTA = 0b00001111; // led 0,1,2,3
    else if (dist_cm <= 6)
        PORTA = 0b00011111; // led 0,1,2,3,4
    else if (dist_cm <= 7)
        PORTA = 0b00111111; // led 0,1,2,3,4,5
    else if (dist_cm <= 10)
        PORTA = 0b01111111; // led 0,1,2,3,4,5,6
    else
        PORTA = 0b11111111; // led 0,1,2,3,4,5,6,7
}

```

4. trigger pulse 하강 edge

```
# main.c
/*
 * 05.ULTRASONIC_MY.c
 *
 * Created: 2026-06-17 오후 3:06:20
 * Author : kccistc
 */

#define F_CPU 16000000UL
#include <avr/io.h>
#include <avr/interrupt.h> // sei, ISR 등
#include <stdio.h>

#include "button.h"

extern void init_button(void);
extern void init_led();
extern int get_button(int button_num, int button_pin);

extern void init_uart0();
extern void UART0_transmit(uint8_t data);
extern void pc_command_processing();
extern void init_ultrasonic(void);
extern void make_trigger(void);
extern void ultrasonic_processing();
extern volatile char scm[50];
extern volatile int echo_done;
extern volatile int ultrasonic_distance;
extern volatile char scm[];
extern void led_distance_control(int dist_cm);

FILE OUTPUT = FDEV_SETUP_STREAM(UART0_transmit, NULL, _FDEV_SETUP_WRITE);

void init_timer0();
volatile uint32_t msec_count = 0; // volatile 의 최적화 방지
volatile int ultrasonic_check_time = 0;

ISR(TIMER0_OVF_vect)
{
    TCNT0 = 6; // TCNT0 6~256: 250 펄스 count 하기 위해
```

```

        msec_count++;
        ultrasonic_check_time++;
    }

int main(void)
{
    init_led();
    init_timer0();
    init_uart0();
    init_ultrasonic();

    stdout = &OUTPUT;          // printf가 동작할 수 있도록 stdout을 설정
    sei();                      // 전역(대문) interrupt 허용

    init_button();

    make_trigger();
    while (1)
    {

        // pc_command_processing();
        ultrasonic_processing();
    }
}

void init_timer0()
{
    // TCNT0 = TimerCount0
    // TCNT0 6 ~ 256 : 250개 펄스를 count 하기 위해 6부터 시작
    TCNT0 = 6;

    TCCR0 &= ~( 1<< CS02 | 1 << CS01 | 1 << CS00); // 0분주 reset 먼저
    TCCR0 |= 1 << CS02 | 0 << CS01 | 0 << CS00;      // 64분주
    TIMSK |= 1 << TOIE0;    // TIMER0 Overflow INT
}

```

```

#ultrasonic.c
/*
 * ultrasonic.c
 *
 * Created: 2026-06-17 오후 1:32:59
 * Author: kccistc
 */

#include "ultrasonic.h"

void init_ultrasonic(void);
void make_trigger(void);
void ultrasonic_processing(void);
void led_distance_control(int dist_cm);

extern volatile int ultrasonic_check_time;

volatile int ultrasonic_distance = 0;
volatile char scm[50];

// p.278 표 12-3
// INT4: PE4 외부(ext) INT4 초음파 센서 상승.하강 엣지 둘다 이곳으로 들어옴
// 결국 상승 엣지 때 한번, 하강 엣지 때 한번 들어옴
// 0x000A

volatile int echo_done = 0; // 하강 엣지 감지 플래그

ISR(INT4_vect)
{
    // 1. 상승 엣지
    if (ECHO_PORT & (1 << ECHO_PIN))
    {
        TCNT1 = 0;

    }
    // 2. 하강 엣지
    else
    {
        // TCNT1: timer count 1
        // ex. TCNT1에 10 들어있다고 가정해보자.
        // 15.625KHz의 1주기: 64μs
        // 0.000064sec * 10 => 0.00064sec(640μs)
        // 640μs/58 (1cm 이동하는데 소요되는 시간) : 11cm
    }
}

```

```

        // 1sec: 1000000μs

        ultrasonic_distance = TCNT1 * 1000000.0 * 1024/ F_CPU;

        // ---- 소요 시간을 cm로 환산 ----
        sprintf((char *)scm, "dis: %dcm\n", ultrasonic_distance/58);
        echo_done = 1; // 플래그 세우기만!
    }

}

void init_ultrasonic(void)
{
    TRIG_DDR |= 1 << TRIG_PIN;          // output mode로 설정
    ECHO_DDR &= ~(1 << ECHO_PIN); // input mode로 설정

    // p.289 표 12-6 p.288 그림 12-8 참조
    // 0 1: 상승에지, 하강에지 둘다 INT 를 띄우도록 요청한다.
    EICRB |= 0 << ISC41 | 1 << ISC40;

    // 16bit timer/counter 1번을 사용하기로 하자.
    // timer int를 사용하지 않는다.
    // 16bit timer1 16bit로 표시할 수 있는 최대값 65535(max)
    // 16MHz / 1024 분주: 16000000Hz/1024 --> 15625Hz -> 15.625KHz
    // 1주기(1개의 필수 소요시간) : T=1/f = 1/15625 --> 0.000064sec => 64μs
    // 분주비 1024로 설정, p.318 표 14-1
    TCCR1B |= 1 << CS12 | 1 << CS10;      // 1024분주

    //----EINT4(external interrupt 4번) 설정----
    // p.287 그림 12-6
    EIMSK |= 1 << INT4;          // 외부 INT4(ECHO핀)

}

void make_trigger(void)
{
    TRIG_PORT &= ~(1 << TRIG_PIN); // low로 만든다.
    _delay_us(1);
    TRIG_PORT |= (1 << TRIG_PIN);  // high로 만든다.
    _delay_us(15);                  //          규격에는          10μs인데
    redundancy로
    TRIG_PORT &= ~(1 << TRIG_PIN); // low로 만든다.

```

```
}
```

```
void ultrasonic_processing(void)
```

```
{
```

```
    if (echo_done == 1)    // 1초
```

```
    {
```

```
        echo_done = 0;
```

```
        // printf("%s", scm)
```

```
        ultrasonic_check_time = 0;    // 초기화
```

```
        printf("%s", scm);
```

```
        make_trigger();
```

```
        int dist_cm = ultrasonic_distance / 58;
```

```
        led_distance_control(dist_cm);
```

```
    }
```

```
}
```

```
void led_distance_control(int dist_cm)
```

```
{
```

```
    if (dist_cm <= 2)
```

```
        PORTA = 0b00000001;    // led 0
```

```
    else if (dist_cm <= 3)
```

```
        PORTA = 0b00000011;    // led 0,1
```

```
    else if (dist_cm <= 4)
```

```
        PORTA = 0b00000111;    // led 0,1,2
```

```
    else if (dist_cm <= 5)
```

```
        PORTA = 0b00001111;    // led 0,1,2,3
```

```
    else if (dist_cm <= 6)
```

```
        PORTA = 0b00011111;    // led 0,1,2,3,4
```

```
    else if (dist_cm <= 7)
```

```
        PORTA = 0b00111111;    // led 0,1,2,3,4,5
```

```
    else if (dist_cm <= 10)
```

```
        PORTA = 0b01111111;    // led 0,1,2,3,4,5,6
```

```
    else
```

```
        PORTA = 0b11111111;    // led 0,1,2,3,4,5,6,7
```

```
}
```